DevOps Lab Assessment - Experiment 7

Docker Networking: Inter-Container IP & DNS Name Communication

COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ASSESSMENT:**	Question 7 (5 Marks)
TOPIC:	Docker Networking & DNS Resolution	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Containerized microservices must communicate reliably across isolated virtual networks. The objective of this experiment is to evaluate Docker networking modes, demonstrating: (1) the architectural limitations of the default bridge network regarding name resolution, (2) the creation of a user-defined bridge network, (3) inter-container communication using raw IP addresses, and (4) automatic service discovery and inter-container communication using container hostnames via Docker's embedded DNS engine.

2. Docker Networking Architecture & Drivers

Docker creates virtual Ethernet bridge interfaces (e.g. docker0) on the host. Crucially, the **default bridge** network does *not* provide automatic DNS resolution; containers can only reach each other if they know each other's ephemeral IP addresses. Conversely, **user-defined bridge networks** enable an embedded DNS server listening at 127.0.0.11:53, allowing seamless resolution of container names.

Network Type	Driver	IP Communication	Automatic DNS Resolution	Network Isolation
Default Bridge	bridge	Supported	No (Requires legacy --link)	Shared by all default containers
User-Defined Bridge	bridge	Supported	Yes (via 127.0.0.11)	Isolated namespace boundary
Host Network	host	Shares host IP	Uses host /etc/resolv.conf	None (bypasses Docker NAT)

3. Experiment 7.1: Inspecting Default Bridge Network Limitations

```
$ # 1. Run two containers on default bridge docker run -d --name c1-server alpine sleep 3600 docker run -d --name c2-client alpine sleep 3600 # 2. Retrieve c1 IP address C1_IP=$(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' c1-server) echo "Server IP: $C1_IP" # 3. Ping by IP address (Expected: SUCCESS) docker exec c2-client ping -c 2 $C1_IP # 4. Ping by Container Name (Expected: FAILURE) docker exec c2-client ping -c 1 c1-server
Server IP: 172.17.0.2 PING 172.17.0.2 (172.17.0.2): 56 data bytes 64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.082 ms 64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.071 ms ping: bad address 'c1-server' OBSERVATION: Name resolution failed! Default bridge lacks internal DNS server.
```

4. Experiment 7.2: Creating and Inspecting Custom Bridge Network

```
$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-custom-net docker network inspect lab-custom-net --format "Subnet: {{range .IPAM.Config}}{{.Subnet}}{{end}} | Gateway: {{range .IPAM.Config}}{{.Gateway}}{{end}}"
4a8b7c9120e543b12... Subnet: 172.20.0.0/16 | Gateway: 172.20.0.1
```

```
$ # Clean up old containers and launch on custom network docker rm -f c1-server c2-client docker run -d --name backend-server --network lab-custom-net alpine sh -c \ "while true; do echo -e 'HTTP/1.1 200 OK\r\n\r\nHello from Backend Server' | nc -l -p 8080; done" docker run -d --name frontend-client --network lab-custom-net alpine sleep 3600
backend-server (started on lab-custom-net, IP: 172.20.0.2) frontend-client (started on lab-custom-net, IP: 172.20.0.3)
```

5. Experiment 7.3 & 7.4: Verification of IP vs Name Resolution

5.1 Communication via Raw IP Address (Experiment 7.3)

```
$ docker exec frontend-client wget -qO- http://172.20.0.2:8080
Hello from Backend Server
```

5.2 Communication via Container Hostname & Embedded DNS (Experiment 7.4)

```
$ docker exec frontend-client wget -qO- http://backend-server:8080
Hello from Backend Server
```

5.3 DNS Inspection via nslookup

```
$ docker exec frontend-client nslookup backend-server
Server: 127.0.0.11 Address: 127.0.0.11:53 Name: backend-server Address: 172.20.0.2
```

6. Engineering Observations & Conclusion

Key Technical Takeaways

1. **Embedded DNS (127.0.0.11):** Custom networks activate Docker's internal DNS daemon, resolving container names dynamically without static IP hardcoding.
2. **Resilience to IP Churn:** If backend-server restarts and receives a new IP address, frontend-client continues communicating via the unchanging container name.
3. **Security Isolation:** Containers residing on distinct user-defined bridges cannot communicate directly, establishing strict network segmentation.